

Coding Principles and Architecture Patterns

- Pages compose features together

1. Clean Code

6. Reusable Components

Clean components/easy to read, easy to maintain, and easy to extend.
Create components that can be used in multiple places.

Guidelines

Example

- Use meaningful names.

```
<Button label="Get Started" variant="primary" />
```

```
<InputText id="email" label="Email" />
```

Better

- Write code that explains itself

7. Proper Folder Structure

Example

Proper folder structure makes the project easier to navigate.

```
function formatUserName(firstName, lastName) {
```

Example

```
  pages/ ${firstName} ${lastName}.trim();
```

```
src/  
  app/  
    shared/  
      components/  
        ui/  
          Button.jsx  
          shared/  
            text.jsx  
            utils/
```

2. DRY (Don't Repeat Yourself)

Avoid duplicating logic across the project.

```
shared/  
  text.jsx  
  utils/
```

Example

Instead of repeating the same button styling in multiple files, create a reusable component.

8. Consistent Naming

3. Setup Principles

- Keep components focused on one job.
Use consistent naming across files, functions, variables, and folders.
A package may contain folders instead of duplicating markup.

Rules: routing and app-level setup in the app folder.

- S: Single Responsibility Principle

4. Use a single level of organization for related UI.

- O: Open/Closed Principle

5. Use Pascal Case for UI components.

- L: Liskov Substitution Principle

6. Use lowercase folder names for feedback UI.

- I: Interface Segregation Principle

9. Dependency Inversion Principle

9. Type Safety

Prefer strong typing where possible.

Example

In this project, a button component should only handle button rendering and behavior, not page-specific logic.

Reusable component use no validation.

4. FSD / Feature Sliced Design

- Future improvements can include TypeScript.
Organize code by feature, not by technical layer only.

10. Error Handling

Suggested structure

Handle errors gracefully so the app does not break silently.

```
src/
```

Example

```
try  
  pages/  
    widgets/  
      saveData();  
  } catch (error) {  
    console.error('Failed to save data', error);  
  } shared/
```

11. Testing Basics

app: routing and app shell

Testing helps protect the app from regressions.

- pages: route-level pages

Example

- shared/ui: reusable UI components

- Unit tests for UI components

5. Separation of Concerns

Each module should focus on one responsibility.

Illustration: Good vs Bad Structure

Example

Bad

- UI components render UI

